Alternative Assessment Tool (AAT) Report on

"Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)
[Course Code: 23ECSPE1AD]

Submitted by

SHWETA K **1BM24EC204**

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

2026 – 2027

Question 1: Arithmetic Logic Unit (ALU)

Problem Statement

Design an 8-bit Arithmetic Logic Unit (ALU) in Verilog HDL. The ALU takes two 8-bit operands A and B and a 4-bit opcode, and produces an 8-bit result together with carry, zero, negative and overflow flags. Write a testbench to verify its operation.

Requirements

- Arithmetic: ADD, SUB, INC, DEC
- Logic: AND, OR, XOR, NOT, NAND, NOR, XNOR
- Shift left / shift right by one bit
- PASS A, PASS B and CLEAR
- Flags: carry (or borrow), zero, negative, signed overflow

Design Circuit Diagram

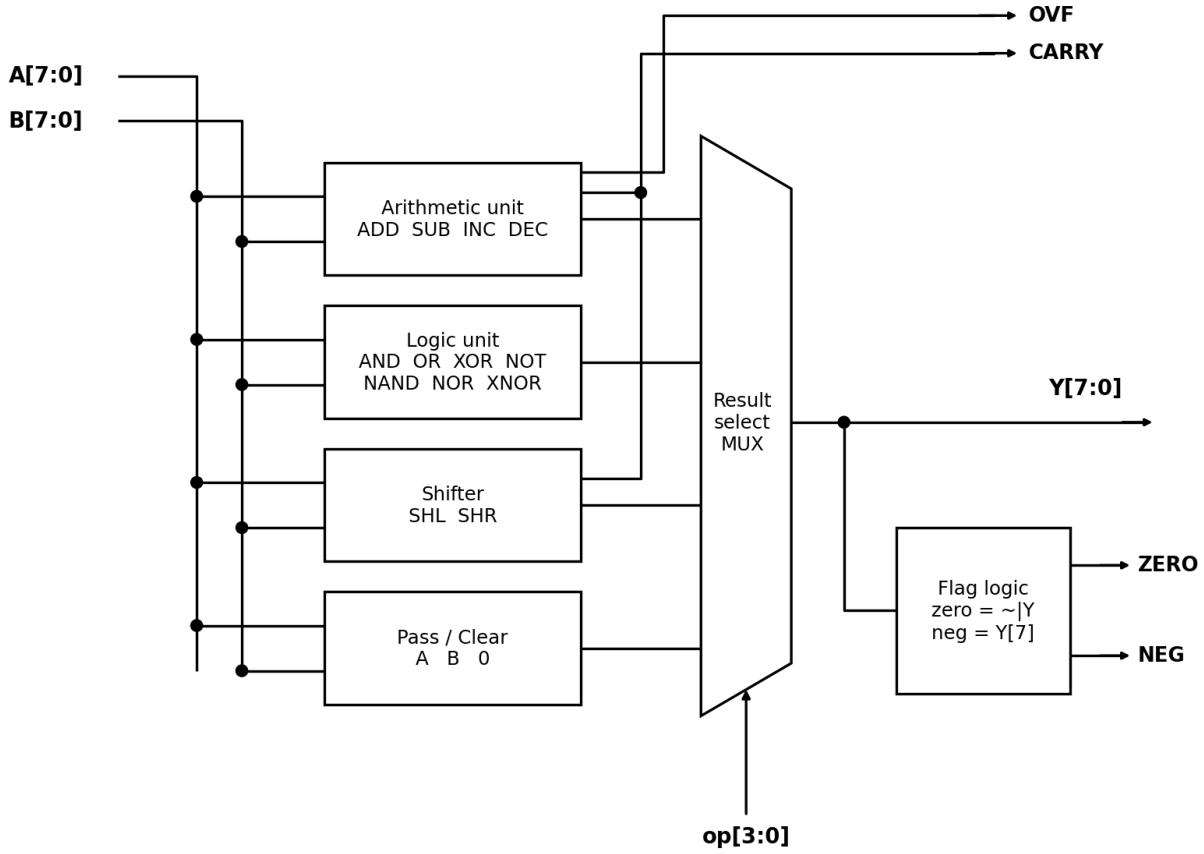


Figure 1: Block diagram of the 8-bit ALU

Function table (op code selects the operation)

Opcode	Operation	Opcode	Operation
0000	Y = A + B	1000	Y = A XNOR B
0001	Y = A - B	1001	Y = A << 1 (carry = A[7])
0010	Y = A AND B	1010	Y = A >> 1 (carry = A[0])
0011	Y = A OR B	1011	Y = A + 1
0100	Y = A XOR B	1100	Y = A - 1
0101	Y = NOT A	1101	Y = A (pass A)
0110	Y = A NAND B	1110	Y = B (pass B)
0111	Y = A NOR B	1111	Y = 0 (clear)

Flag definitions

Flag	Meaning
carry	Carry-out for ADD / INC, borrow for SUB / DEC, bit shifted out for SHL / SHR
zero	1 when all bits of Y are 0
neg	Copy of the MSB of the result, Y[7]
ovf	Signed overflow. ADD: A7 = B7 and Y7 != A7. SUB: A7 != B7 and Y7 != A7

Design (Verilog HDL)

alu.v

```
`timescale 1ns/1ps
//==============================================================================
// Module : alu  (N-bit Arithmetic Logic Unit, default N = 8)
// Type   : combinational
// op map : 0000 ADD     0001 SUB      0010 AND      0011 OR
//          0100 XOR     0101 NOT A    0110 NAND     0111 NOR
//          1000 XNOR    1001 SHL A    1010 SHR A    1011 INC A
//          1100 DEC A   1101 PASS A   1110 PASS B   1111 CLEAR
// Flags  : carry (carry-out for ADD/INC, borrow for SUB/DEC, shifted-out bit
//          for SHL/SHR), zero, neg (MSB of result), ovf (signed overflow,
//          ADD and SUB only)
//==============================================================================
module alu #(parameter N = 8) (
    input  [N-1:0] a,
    input  [N-1:0] b,
    input  [3:0]   op,
    output reg [N-1:0] y,
    output reg         carry,
    output             zero,
    output             neg,
    output reg         ovf
);
    reg [N:0] t;

    always @(*) begin
        t     = {(N+1){1'b0}};
        y     = {N{1'b0}};
        carry = 1'b0;
        ovf   = 1'b0;
        case (op)
            4'b0000: begin                          // ADD
                t     = {1'b0, a} + {1'b0, b};
                y     = t[N-1:0];
                carry = t[N];
                ovf   = (a[N-1] == b[N-1]) & (y[N-1] != a[N-1]);
            end
            4'b0001: begin                          // SUB  (a - b)
                t     = {1'b0, a} - {1'b0, b};
                y     = t[N-1:0];
                carry = t[N];                       // borrow
                ovf   = (a[N-1] != b[N-1]) & (y[N-1] != a[N-1]);
            end
            4'b0010: y = a & b;                     // AND
            4'b0011: y = a | b;                     // OR
            4'b0100: y = a ^ b;                     // XOR
            4'b0101: y = ~a;                        // NOT A
```

```
            4'b0110: y = ~(a & b);               // NAND
            4'b0111: y = ~(a | b);               // NOR
            4'b1000: y = ~(a ^ b);               // XNOR
            4'b1001: begin y = a << 1; carry = a[N-1]; end   // SHL
            4'b1010: begin y = a >> 1; carry = a[0];   end   // SHR
            4'b1011: begin                       // INC A
                t = {1'b0, a} + {{N{1'b0}}, 1'b1};
                y = t[N-1:0]; carry = t[N];
            end
            4'b1100: begin                       // DEC A
                t = {1'b0, a} - {{N{1'b0}}, 1'b1};
                y = t[N-1:0]; carry = t[N];
            end
            4'b1101: y = a;                      // PASS A
            4'b1110: y = b;                      // PASS B
            default: y = {N{1'b0}};              // CLEAR
        endcase
    end

    assign zero = (y == {N{1'b0}});
    assign neg  = y[N-1];
endmodule
```

Testbench

alu_tb.v

```
`timescale 1ns/1ps
module alu_tb;
    reg  [7:0] a, b;
    reg  [3:0] op;
    wire [7:0] y;
    wire carry, zero, neg, ovf;
    integer i;

    alu dut (.a(a), .b(b), .op(op), .y(y), .carry(carry), .zero(zero), .neg(neg), .ovf(ovf));

    initial begin
        $dumpfile("alu_tb.vcd"); $dumpvars(0, alu_tb);
        $monitor("op=%b a=%h b=%h | y=%h carry=%b zero=%b neg=%b ovf=%b", op, a, b, y, carry, zero, neg,
ovf);

        // try every operation on the same inputs
        a = 8'hF0; b = 8'h3C;
        for (i = 0; i < 16; i = i + 1) begin
            op = i; #10;
        end

        // carry and overflow cases
        a = 8'd200; b = 8'd100; op = 4'b0000; #10;   // ADD with carry
        a = 8'd100; b = 8'd100; op = 4'b0000; #10;   // ADD with overflow
        a = 8'd20;  b = 8'd50;  op = 4'b0001; #10;   // SUB with borrow
        $finish;
    end
endmodule
```

Run and Observe

```
iverilog -o sim alu_tb.v alu.v && vvp sim
```

Every opcode (0 to 15) is applied to A = F0h and B = 3Ch. Extra cases then show a carry (200 + 100), signed overflow (100 + 100) and a borrow (20 - 50).

Result Waveforms

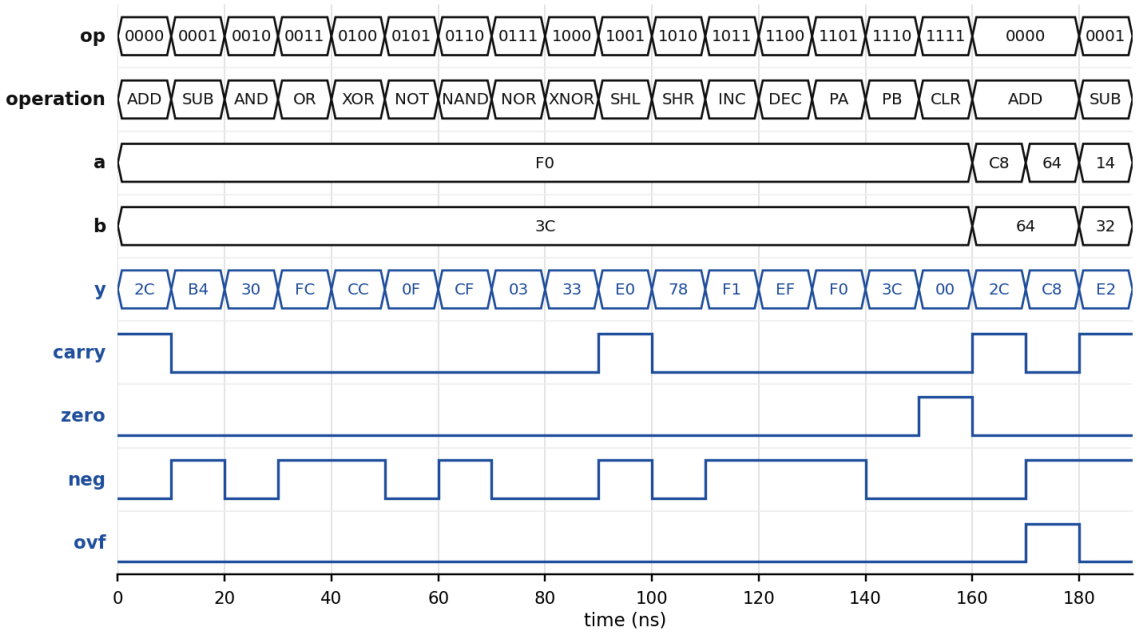


Figure 17: Result waveform of the ALU testbench (plotted from a behavioural model of the design)

With A = F0h and B = 3Ch, op steps through 0000 to 1111 (ADD gives y = 2Ch with carry = 1). The last three steps show a carry (C8h + 64h), an overflow (64h + 64h, ovf = 1) and a borrow (14h - 32h, carry = 1).

Question 2: Comparator

Problem Statement

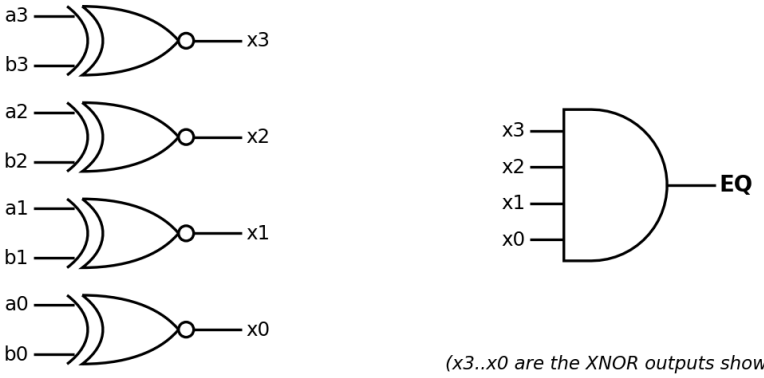
Design a magnitude comparator that compares two 4-bit numbers A and B and generates three outputs: A > B, A = B and A < B. Implement it behaviourally and also at gate level, and write a testbench.

Requirements

- Behavioural version using relational operators
- Gate-level version using XNOR equality terms and AND-OR logic
- Exactly one of GT, EQ, LT is high at any time

Design Circuit Diagram

(a) Equality stage: x_i = XNOR(a_i, b_i) EQ = x3 · x2 · x1 · x0



(x3..x0 are the XNOR outputs shown above)

(b) A > B (GT = a3b3' + x3a2b2' + ...) **(c) A < B (LT = a3'b3 + x3a2'b2 + ...)**

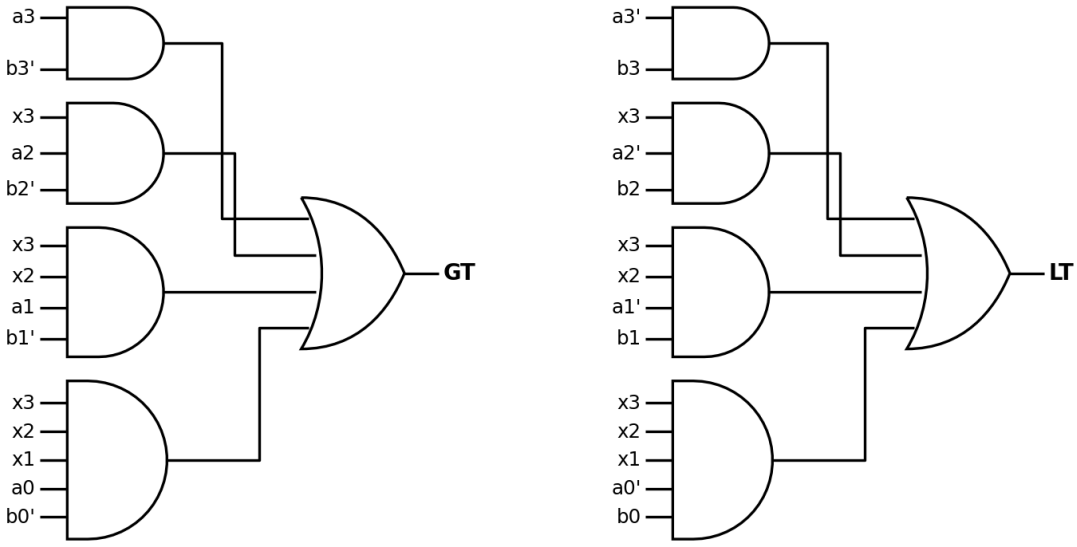


Figure 2: Gate-level design of the 4-bit magnitude comparator

Truth table of a 1-bit comparator (basis of the 4-bit design)

A	B	A > B	A = B	A < B
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

Equations

EQ = x3 x2 x1 x0

GT = a3 b3' + x3 a2 b2' + x3 x2 a1 b1' + x3 x2 x1 a0 b0'

LT = a3' b3 + x3 a2' b2 + x3 x2 a1' b1 + x3 x2 x1 a0' b0

Design (Verilog HDL)

comparator.v

```
`timescale 1ns/1ps
//==============================================================================
// comparator       : behavioural N-bit magnitude comparator (default N = 4)
// comparator_4bit_gate : gate-level 4-bit version
//   x_i = XNOR(a_i, b_i)
//   EQ  = x3 x2 x1 x0
//   GT  = a3 b3' + x3 a2 b2' + x3 x2 a1 b1' + x3 x2 x1 a0 b0'
//   LT  = a3' b3 + x3 a2' b2 + x3 x2 a1' b1 + x3 x2 x1 a0' b0
//==============================================================================
module comparator #(parameter N = 4) (
    input  [N-1:0] a,
    input  [N-1:0] b,
    output         gt,
    output         eq,
    output         lt
);
    assign gt = (a > b);
    assign eq = (a == b);
    assign lt = (a < b);
endmodule

module comparator_4bit_gate (
    input  [3:0] a,
    input  [3:0] b,
    output       gt,
    output       eq,
    output       lt
);
    wire [3:0] x;
    assign x = ~(a ^ b);
    assign eq = &x;
    assign gt = (a[3] & ~b[3])
              | (x[3] & a[2] & ~b[2])
              | (x[3] & x[2] & a[1] & ~b[1])
              | (x[3] & x[2] & x[1] & a[0] & ~b[0]);
    assign lt = (~a[3] & b[3])
              | (x[3] & ~a[2] & b[2])
              | (x[3] & x[2] & ~a[1] & b[1])
              | (x[3] & x[2] & x[1] & ~a[0] & b[0]);
endmodule
```

Testbench

comparator_tb.v

```
`timescale 1ns/1ps
module comparator_tb;
    reg  [3:0] a, b;
    wire gt, eq, lt;


    comparator #(4) dut (.a(a), .b(b), .gt(gt), .eq(eq), .lt(lt));
```

```
    initial begin
        $dumpfile("comparator_tb.vcd"); $dumpvars(0, comparator_tb);
        $monitor("a=%d b=%d | gt=%b eq=%b lt=%b", a, b, gt, eq, lt);

        a = 4; b = 4;  #10;
        a = 9; b = 5;  #10;
        a = 2; b = 12; #10;
        a = 15; b = 0; #10;
        a = 0; b = 15; #10;
        $finish;
    end
endmodule
```

Run and Observe

```
iverilog -o sim comparator_tb.v comparator.v && vvp sim
```

The testbench uses the behavioural module. To test the gate-level version, replace "comparator #(4)" with "comparator_4bit_gate" in the instantiation. For (9,5) GT = 1; for (4,4) EQ = 1; for (2,12) LT = 1.

Result Waveforms

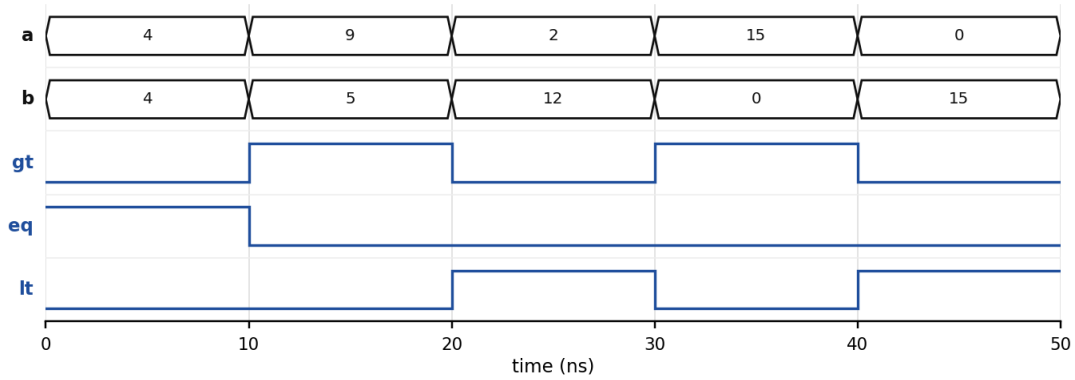


Figure 18: Result waveform of the comparator testbench (plotted from a behavioural model of the design)

gt = 1 for (9, 5) and (15, 0), eq = 1 for (4, 4), and lt = 1 for (2, 12) and (0, 15).

Question 3: 4-Bit Parity Generator and Checker

Problem Statement

Design a 4-bit parity generator that produces a parity bit for a 4-bit data word, and a parity checker that detects whether the received 5-bit word (data + parity) contains an error. Write a testbench for both.

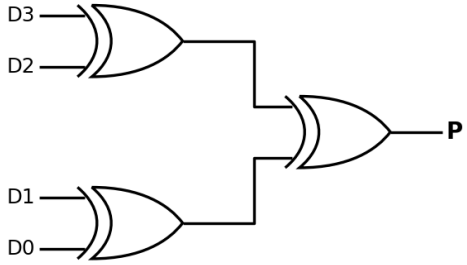
Requirements

- Generator: even parity (odd parity selectable through a parameter)
- Checker: error output is 1 when an odd number of bits are corrupted

Design Circuit Diagram

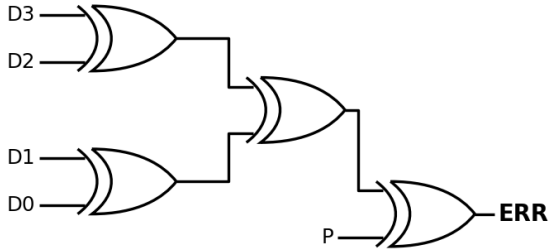
(a) Parity generator (even)

P = D3 ^ D2 ^ D1 ^ D0



(b) Parity checker (even)

ERR = D3 ^ D2 ^ D1 ^ D0 ^ P



For odd parity, invert P (use XNOR in the last stage).

Figure 3: 4-bit parity generator and parity checker (even parity)

Truth table of the parity generator

D3 D2 D1 D0	P (even)	P (odd)	D3 D2 D1 D0	P (even)	P (odd)
0000	0	1	1000	1	0
0001	1	0	1001	0	1
0010	1	0	1010	0	1
0011	0	1	1011	1	0
0100	1	0	1100	0	1
0101	0	1	1101	1	0
0110	0	1	1110	1	0
0111	1	0	1111	0	1

Parity checker (even parity): ERR depends on the number of 1s in D3..D0 and P

Number of 1s in {D3..D0, P}	ERR	Result
Even (0, 2 or 4)	0	No error detected
Odd (1, 3 or 5)	1	Parity error detected

Design (Verilog HDL)

parity.v

```
`timescale 1ns/1ps
//==============================================================================
// parity_gen_4bit : p = d3^d2^d1^d0 (even parity, ODD=0) or its complement
//                   (odd parity, ODD=1)
// parity_chk_4bit : err = d3^d2^d1^d0^p  (^ ODD). err = 1 -> parity error
```

```
//================================================================================
module parity_gen_4bit #(parameter ODD = 0) (
    input  [3:0] d,
    output       p
);
    assign p = (^d) ^ ODD;
endmodule

module parity_chk_4bit #(parameter ODD = 0) (
    input  [3:0] d,
    input        p,
    output       err
);
    assign err = (^{d, p}) ^ ODD;
endmodule
```

Testbench

parity_tb.v

```
`timescale 1ns/1ps
module parity_tb;
    reg  [3:0] d;
    reg        flip;                       // 1 = flip one bit to create an error
    wire       p, err;
    wire [3:0] rx = {d[3:1], d[0] ^ flip}; // data seen by the checker
    integer i;


    parity_gen_4bit #(0) gen (.d(d),  .p(p));            // even parity
    parity_chk_4bit #(0) chk (.d(rx), .p(p), .err(err));


    initial begin
        $dumpfile("parity_tb.vcd"); $dumpvars(0, parity_tb);
        $monitor("data=%b parity=%b flip=%b received=%b | error=%b", d, p, flip, rx, err);


        for (i = 0; i < 16; i = i + 1) begin
            d = i; flip = 0; #5;    // no error    -> err = 0
            flip = 1; #5;           // 1-bit error -> err = 1
        end
        $finish;
    end
endmodule
```

Run and Observe

```
iverilog -o sim parity_tb.v parity.v && vvp sim
```

All 16 data words are applied. With flip = 0 the error output stays 0. With flip = 1 (one data bit inverted) the error output becomes 1.

Result Waveforms

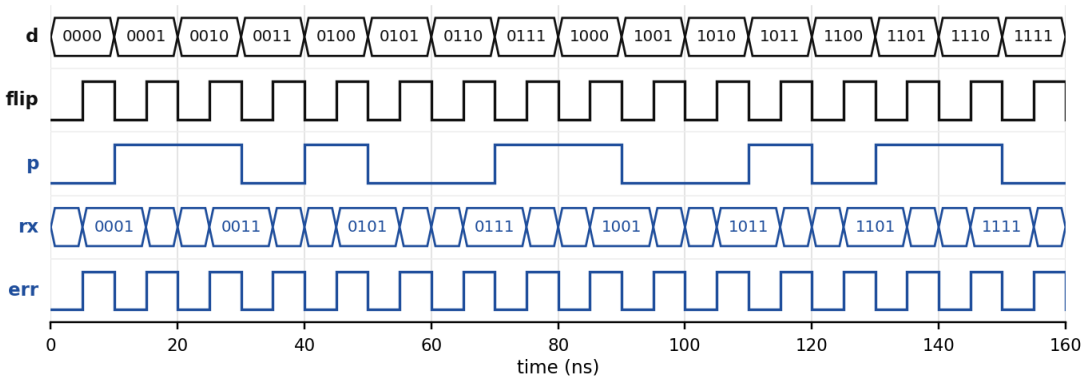


Figure 19: Result waveform of the parity generator and checker testbench (plotted from a behavioural model of the design)

p is the even-parity bit of d. err stays 0 while flip = 0 and becomes 1 whenever flip inverts one data bit.

Question 4: 4-Bit Barrel Shifter

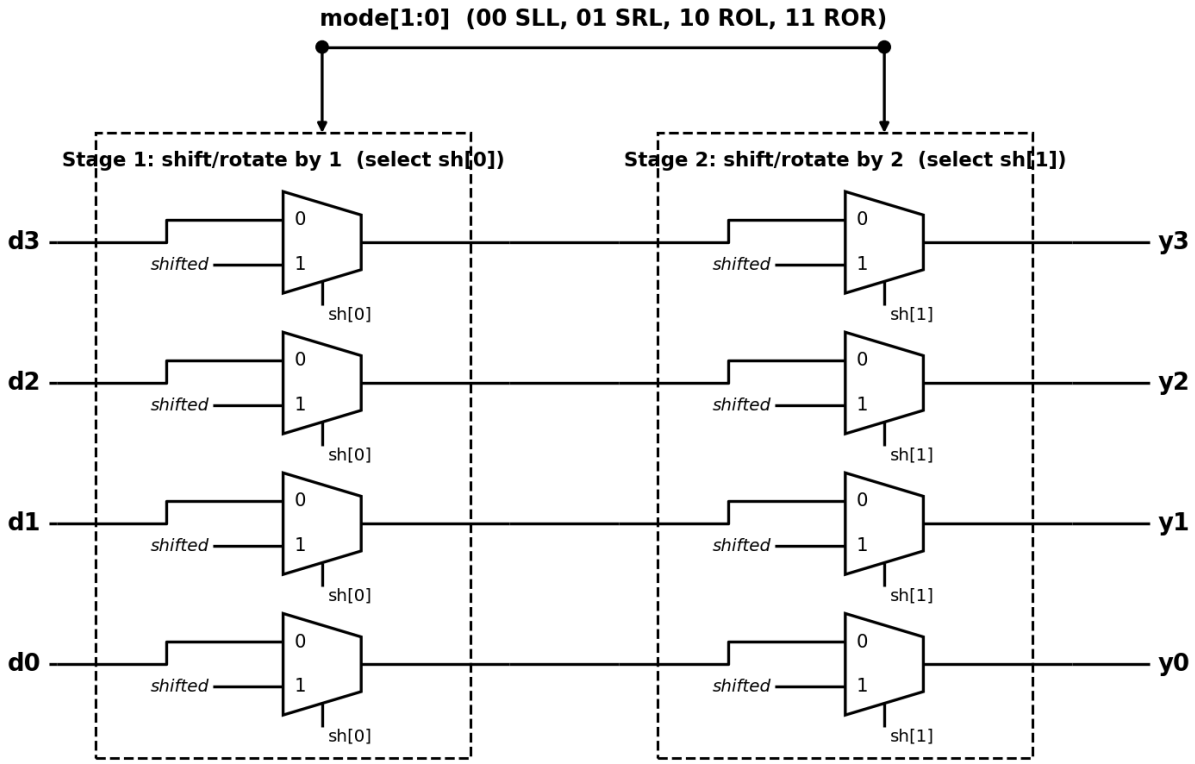
Problem Statement

Design a 4-bit barrel shifter that can shift or rotate a 4-bit input by 0 to 3 positions in a single combinational step, and write a testbench.

Requirements

- Modes: logical shift left, logical shift right, rotate left, rotate right
- Shift amount selectable from 0 to 3
- Built from two multiplexer stages (shift by 1, then shift by 2)

Design Circuit Diagram



'shifted' input = d(i-1) for left, d(i+1) for right; the vacated bit is 0 (shift) or the bit that fell off (rotate)

Figure 4: Two-stage 4-bit barrel shifter

Mode select

mode[1:0]	Operation
00	Logical shift left (SLL), zeros fill from the right
01	Logical shift right (SRL), zeros fill from the left
10	Rotate left (ROL)
11	Rotate right (ROR)

Example: output y for d = 1011

Shift amount sh	SLL (00)	SRL (01)	ROL (10)	ROR (11)
0	1011	1011	1011	1011
1	0110	0101	0111	1101
2	1100	0010	1110	1110
3	1000	0001	1101	0111

Design (Verilog HDL)

barrel_shifter.v

```
`timescale 1ns/1ps
//==============================================================================
// barrel_shifter_4bit : 2-stage mux-based barrel shifter
//   mode[1:0] : 00 logical shift left   01 logical shift right
//               10 rotate left          11 rotate right
//   sh[1:0]   : shift amount 0..3  (stage 1 shifts by 1, stage 2 by 2)
//==============================================================================
module barrel_shifter_4bit (
    input  [3:0] d,
    input  [1:0] sh,
    input  [1:0] mode,
    output [3:0] y
);
    wire right = mode[0];
    wire rot   = mode[1];

    // ---- stage 1 : shift/rotate by 1 when sh[0] = 1
    wire [3:0] s1_l = rot ? {d[2:0], d[3]} : {d[2:0], 1'b0};
    wire [3:0] s1_r = rot ? {d[0], d[3:1]} : {1'b0, d[3:1]};
    wire [3:0] s1   = sh[0] ? (right ? s1_r : s1_l) : d;

    // ---- stage 2 : shift/rotate by 2 when sh[1] = 1
    wire [3:0] s2_l = rot ? {s1[1:0], s1[3:2]} : {s1[1:0], 2'b00};
    wire [3:0] s2_r = rot ? {s1[1:0], s1[3:2]} : {2'b00, s1[3:2]};

    assign y = sh[1] ? (right ? s2_r : s2_l) : s1;
endmodule
```

Testbench

barrel_shifter_tb.v

```
`timescale 1ns/1ps
module barrel_shifter_tb;
    reg  [3:0] d;
    reg  [1:0] sh, mode;
    wire [3:0] y;
    integer i, j;

    barrel_shifter_4bit dut (.d(d), .sh(sh), .mode(mode), .y(y));

    initial begin
        $dumpfile("barrel_shifter_tb.vcd"); $dumpvars(0, barrel_shifter_tb);
        $monitor("mode=%b (00=SLL 01=SRL 10=ROL 11=ROR) shift=%d d=%b | y=%b", mode, sh, d, y);

        d = 4'b1011;
        for (i = 0; i < 4; i = i + 1)
            for (j = 0; j < 4; j = j + 1) begin
                mode = i; sh = j; #10;
            end
        $finish;
    end
endmodule
```

Run and Observe

```
iverilog -o sim barrel_shifter_tb.v barrel_shifter.v && vvp sim
```

With d = 1011: SLL by 1 gives 0110, SRL by 2 gives 0010, ROL by 3 gives 1101, ROR by 1 gives 1101.

Result Waveforms

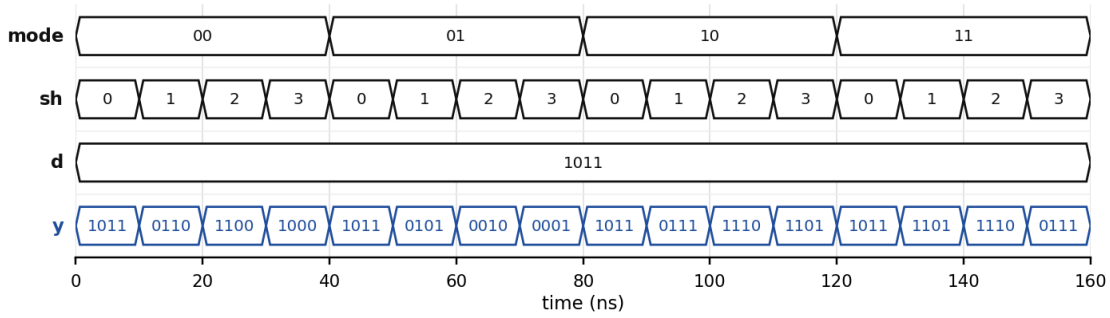


Figure 20: Result waveform of the barrel shifter testbench (plotted from a behavioural model of the design)

For d = 1011, y follows mode and sh. For example, mode 00 with sh = 1 gives 0110, and mode 10 with sh = 3 gives 1101.

Question 5: Timer Module

Problem Statement

Design a programmable one-shot timer. When start is applied, the timer loads a programmed count and counts down once per clock. A busy output stays high while counting, and a done output pulses for one clock when the time expires. Write a testbench.

Requirements

- 8-bit programmable load value
- busy high for exactly "load" clock cycles
- done is a one-cycle pulse at the end
- Asynchronous reset aborts a running count

Design Circuit Diagram

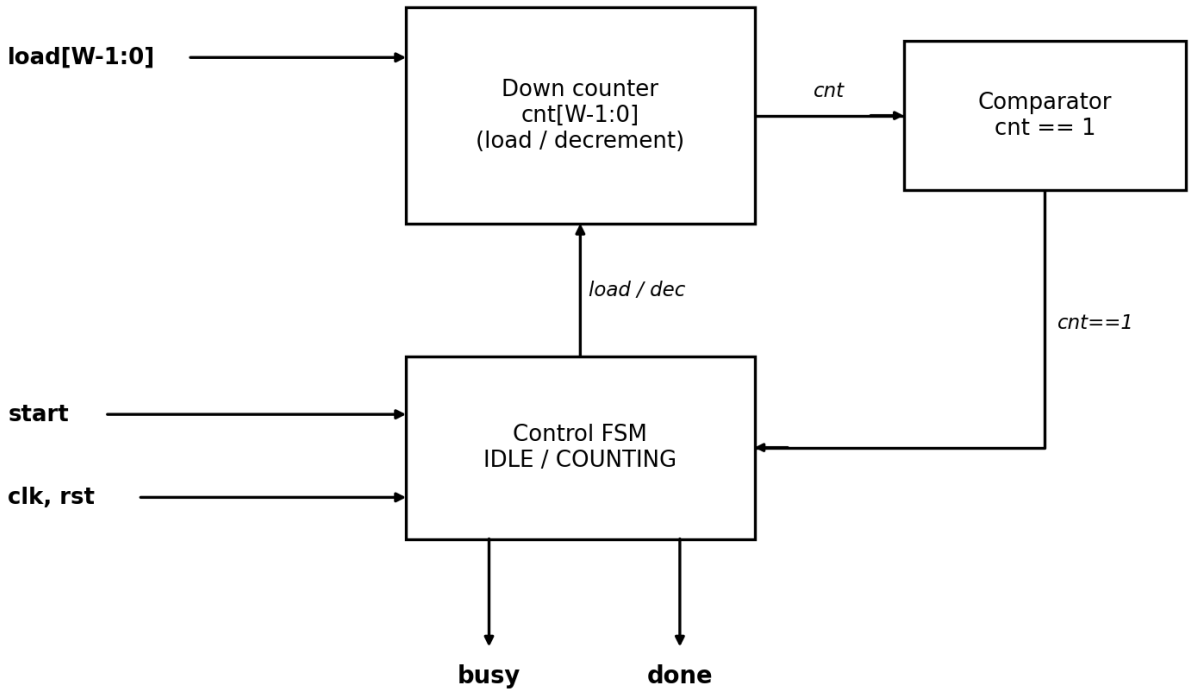


Figure 5: Block diagram of the timer

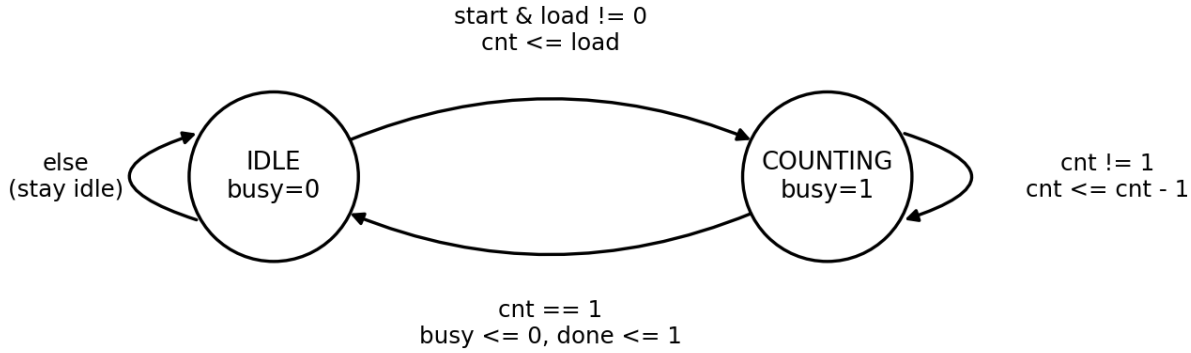


Figure 6: State diagram of the timer control FSM

State table (busy and done are the values after the clock edge)

Present state	Condition	Next state	busy	done	Counter action
IDLE	start = 1 and load != 0	COUNTING	1	0	cnt <= load
IDLE	otherwise	IDLE	0	0	hold
COUNTING	cnt != 1	COUNTING	1	0	cnt <= cnt - 1
COUNTING	cnt = 1	IDLE	0	1	cnt <= 0

Design (Verilog HDL)

timer.v

```
`timescale 1ns/1ps
//==============================================================================
// timer : programmable one-shot down-counter timer
//   start  : pulse high (while idle) to load 'load' and begin counting
//   busy   : high for exactly 'load' clock cycles
//   done   : one-cycle pulse when the programmed time has elapsed
//   load=0 is ignored.  rst is asynchronous and aborts any running count.
//==============================================================================
module timer #(parameter W = 8) (
    input              clk,
    input              rst,
    input              start,
    input  [W-1:0]     load,
    output reg         busy,
    output reg         done
);
    reg [W-1:0] cnt;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            cnt  <= {W{1'b0}};
            busy <= 1'b0;
            done <= 1'b0;
        end else begin
            done <= 1'b0;                          // default: pulse only
            if (!busy) begin
                if (start && (load != {W{1'b0}})) begin
                    cnt  <= load;
                    busy <= 1'b1;
                end
            end else begin
                if (cnt == {{(W-1){1'b0}}, 1'b1}) begin   // cnt == 1
                    busy <= 1'b0;
                    done <= 1'b1;
                    cnt  <= {W{1'b0}};
                end else
                    cnt <= cnt - 1'b1;
            end
        end
    end
endmodule
```

Testbench

timer_tb.v

```
`timescale 1ns/1ps
module timer_tb;
    reg        clk, rst, start;
    reg  [7:0] load;
    wire       busy, done;

    timer dut (.clk(clk), .rst(rst), .start(start), .load(load), .busy(busy), .done(done));

    initial clk = 0;
    always #5 clk = ~clk;

    initial begin
        $dumpfile("timer_tb.vcd"); $dumpvars(0, timer_tb);
        $monitor("t=%0t start=%b load=%d | busy=%b done=%b", $time, start, load, busy, done);
```

```
        rst = 1; start = 0; load = 0;
        #12 rst = 0;

        // run a 5-clock timer
        #10 load = 5; start = 1;
        #10 start = 0;
        #100;

        // run a 3-clock timer
        load = 3; start = 1;
        #10 start = 0;
        #60;
        $finish;
    end
endmodule
```

Run and Observe

```
iverilog -o sim timer_tb.v timer.v && vvp sim
```

A load of 5 keeps busy high for 5 clock cycles, followed by a one-clock done pulse. The same is then repeated for a load of 3.

Result Waveforms

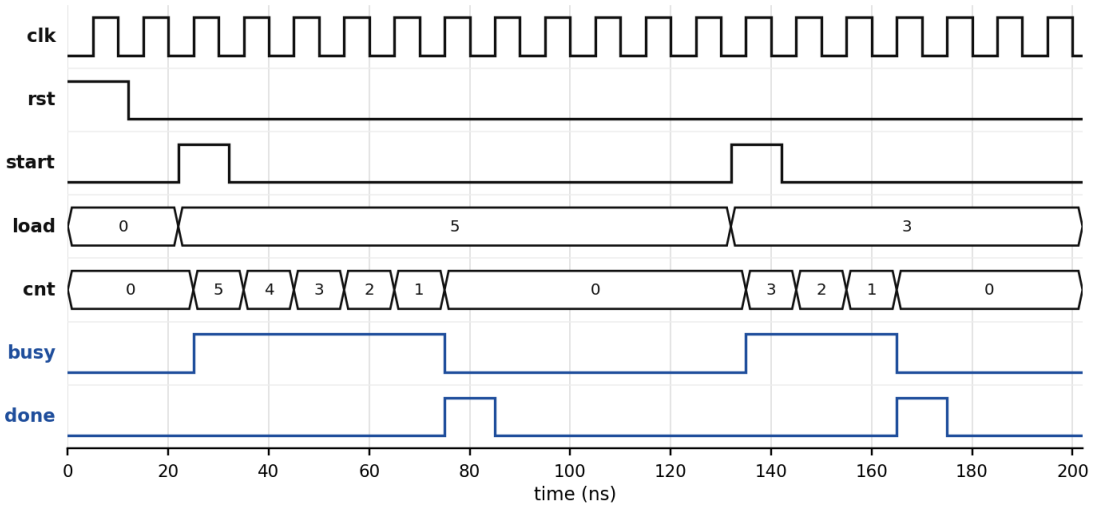


Figure 21: Result waveform of the timer testbench (plotted from a behavioural model of the design)

busy stays high for 5 clock cycles when load = 5 and for 3 clock cycles when load = 3. done pulses for one clock cycle (at 75 ns and 165 ns) when the count ends.

Question 6: Ring Counter

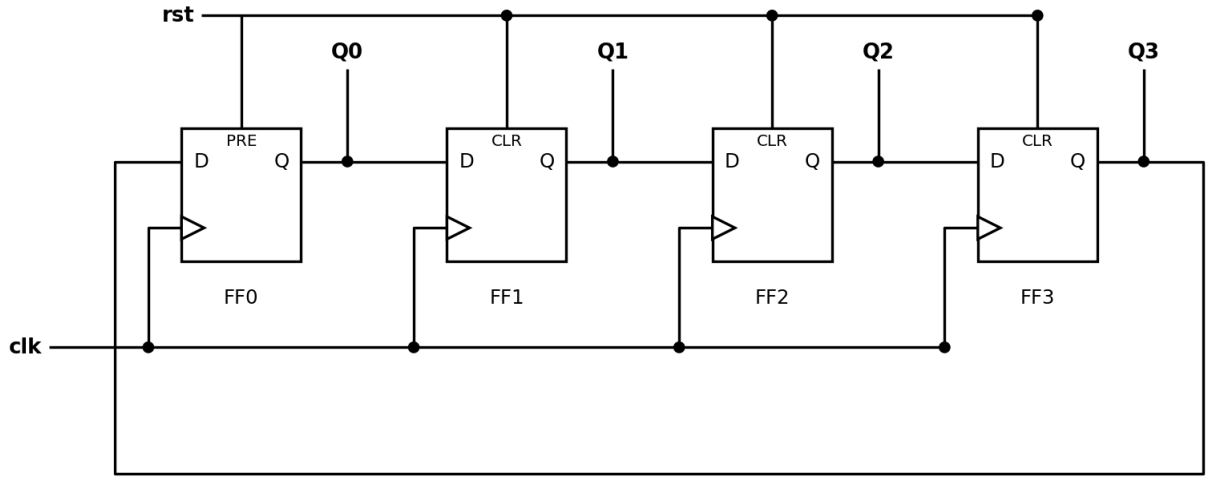
Problem Statement

Design a 4-bit ring counter in which a single 1 circulates through the flip-flops. Write a testbench to show its state sequence.

Requirements

- Reset state 0001
- Last flip-flop output fed back to the first flip-flop

Design Circuit Diagram



Q3 fed back to D0

Figure 7: 4-bit ring counter

State table

Clock pulse	Present state Q3 Q2 Q1 Q0	Next state Q3 Q2 Q1 Q0
0	0001	0010
1	0010	0100
2	0100	1000
3	1000	0001

Design (Verilog HDL)

ring_counter.v

```
`timescale 1ns/1ps
//================================================================================
// ring_counter : N-bit one-hot ring counter (default 4). Reset -> 0001.
// Sequence (N=4): 0001 -> 0010 -> 0100 -> 1000 -> 0001 ...   (N states)
//================================================================================
module ring_counter #(parameter N = 4) (
    input              clk,
    input              rst,
    output reg [N-1:0] q
);
    always @(posedge clk or posedge rst) begin
        if (rst) q <= {{(N-1){1'b0}}, 1'b1};
        else     q <= {q[N-2:0], q[N-1]};
    end
endmodule
```

Testbench

ring_counter_tb.v

```
`timescale 1ns/1ps
module ring_counter_tb;
    reg        clk, rst;
    wire [3:0] q;

    ring_counter #(4) dut (.clk(clk), .rst(rst), .q(q));

    initial clk = 0;
    always #5 clk = ~clk;

    initial begin
        $dumpfile("ring_counter_tb.vcd"); $dumpvars(0, ring_counter_tb);
        $monitor("t=%0t rst=%b q=%b", $time, rst, q);

        rst = 1;
        #12 rst = 0;
        #120;
        $finish;
    end
endmodule
```

Run and Observe

```
iverilog -o sim ring_counter_tb.v ring_counter.v && vvp sim
```

After reset the output cycles 0001, 0010, 0100, 1000 and then repeats (4 states).

Result Waveforms

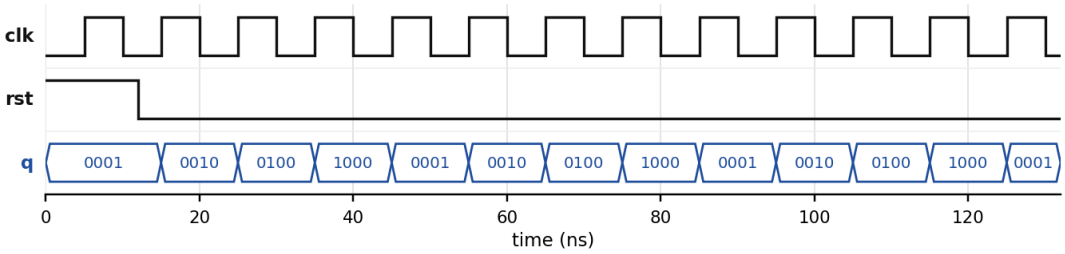


Figure 22: Result waveform of the ring counter testbench (plotted from a behavioural model of the design)

q moves 0001, 0010, 0100, 1000 on successive rising clock edges and then repeats.

Question 7: Johnson Counter

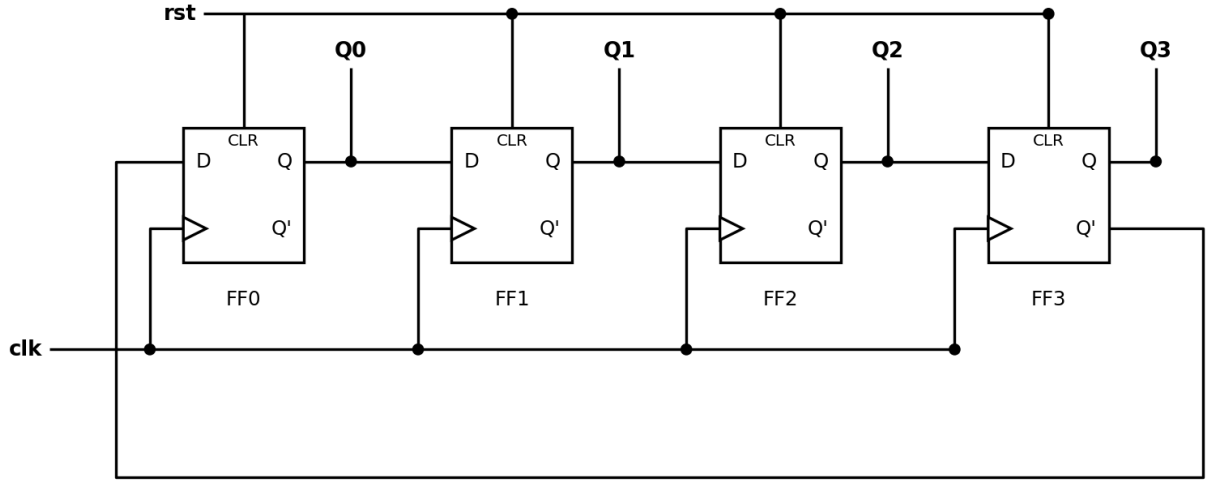
Problem Statement

Design a 4-bit Johnson (twisted-ring) counter, in which the inverted output of the last flip-flop is fed back to the first. Write a testbench to show its state sequence.

Requirements

- Reset state 0000
- 2N = 8 distinct states for N = 4

Design Circuit Diagram



Q3' fed back to D0 (inverted feedback)

Figure 8: 4-bit Johnson counter

State table

Clock pulse	Present state Q3 Q2 Q1 Q0	Next state Q3 Q2 Q1 Q0
0	0000	0001
1	0001	0011
2	0011	0111
3	0111	1111
4	1111	1110
5	1110	1100
6	1100	1000
7	1000	0000

Design (Verilog HDL)

johnson_counter.v

```
`timescale 1ns/1ps
//================================================================================
// johnson_counter : N-bit twisted-ring (Johnson) counter. Reset -> 0000.
// Sequence (N=4): 0000 0001 0011 0111 1111 1110 1100 1000 (2N states)
//================================================================================
module johnson_counter #(parameter N = 4) (
    input              clk,
    input              rst,
    output reg [N-1:0] q
);
    always @(posedge clk or posedge rst) begin
        if (rst) q <= {N{1'b0}};
        else     q <= {q[N-2:0], ~q[N-1]};
```

```
    end
endmodule
```

Testbench

johnson_counter_tb.v

```
`timescale 1ns/1ps
module johnson_counter_tb;
    reg        clk, rst;
    wire [3:0] q;

    johnson_counter #(4) dut (.clk(clk), .rst(rst), .q(q));

    initial clk = 0;
    always #5 clk = ~clk;

    initial begin
        $dumpfile("johnson_counter_tb.vcd"); $dumpvars(0, johnson_counter_tb);
        $monitor("t=%0t rst=%b q=%b", $time, rst, q);

        rst = 1;
        #12 rst = 0;
        #170;
        $finish;
    end
endmodule
```

Run and Observe

```
iverilog -o sim johnson_counter_tb.v johnson_counter.v && vvp sim
```

The output cycles 0000, 0001, 0011, 0111, 1111, 1110, 1100, 1000 and then repeats (8 states).

Result Waveforms

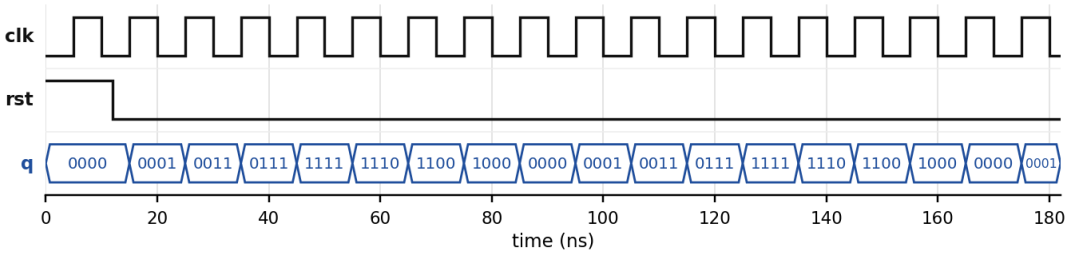


Figure 23: Result waveform of the Johnson counter testbench (plotted from a behavioural model of the design)

q steps through 0000, 0001, 0011, 0111, 1111, 1110, 1100, 1000 and then repeats.

Question 8: Synchronous and Asynchronous Counters

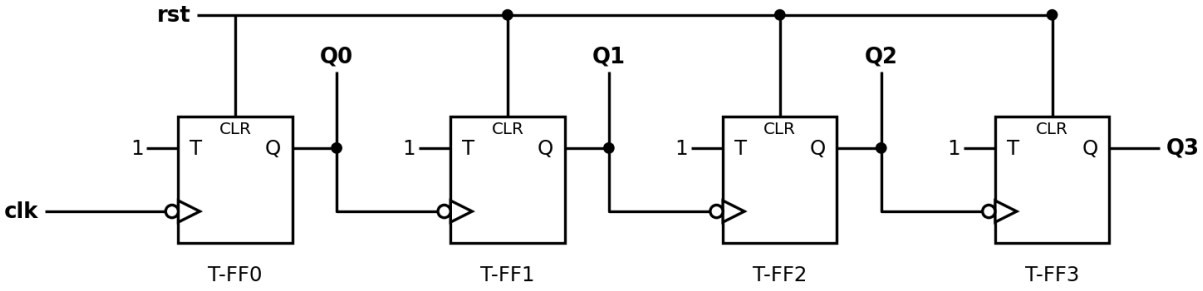
Problem Statement

Design a 4-bit synchronous up/down counter and a 4-bit asynchronous (ripple) up counter, and write a testbench that runs both.

Requirements

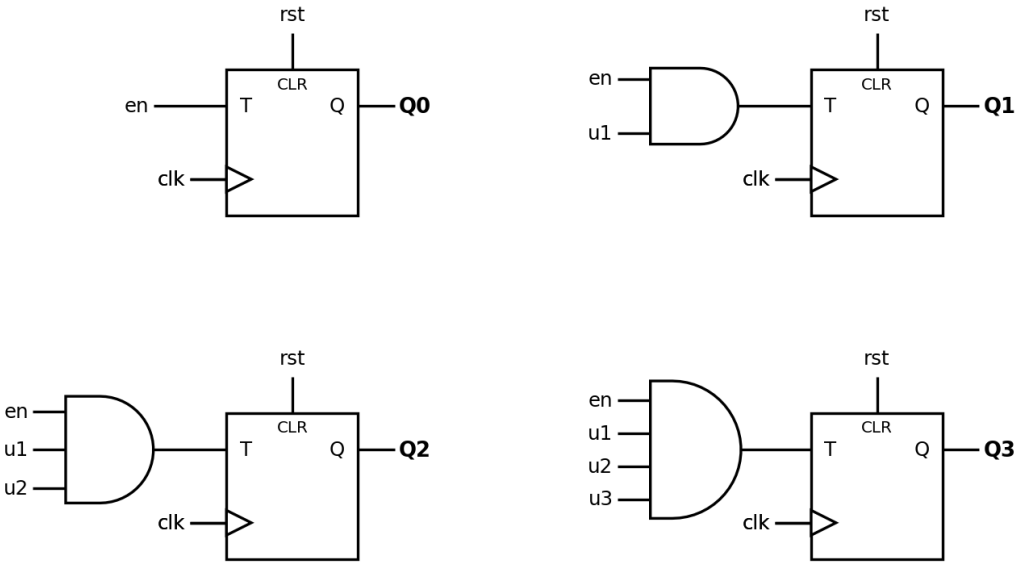
- Synchronous: all flip-flops share one clock; enable and up/down control inputs
- Asynchronous: negative-edge T flip-flops, where each stage is clocked by the output of the previous stage
- Both counters reset to 0

Design Circuit Diagram



Each T flip-flop (T = 1) toggles on the falling edge of its clock; the clock of FFi+1 is Qi

Figure 9: 4-bit asynchronous (ripple) counter using T flip-flops



u_i = XNOR(up, Q_{i-1}) (= Q_{i-1} when counting up, Q_{i-1}' when counting down)

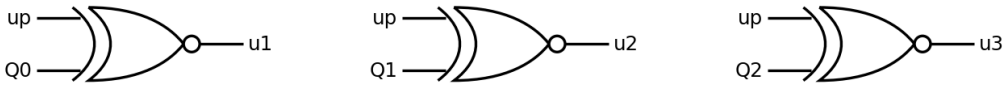


Figure 10: 4-bit synchronous up/down counter using T flip-flops

Control table of the synchronous counter

en	up	Action on the clock edge
0	X	Hold (no change)
1	1	Count up
1	0	Count down

Toggle inputs of the synchronous counter (u_i = up XNOR Q(i-1))

Flip-flop	Toggle input
T0	en
T1	en . u1
T2	en . u1 . u2
T3	en . u1 . u2 . u3

Count sequence of the asynchronous counter

Clock pulse	Q3 Q2 Q1 Q0	Decimal	Clock pulse	Q3 Q2 Q1 Q0	Decimal
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	10	1010	10
3	0011	3	11	1011	11
4	0100	4	12	1100	12
5	0101	5	13	1101	13
6	0110	6	14	1110	14
7	0111	7	15	1111	15

Design (Verilog HDL)

counters.v

```
`timescale 1ns/1ps
//==============================================================================
// sync_counter_4bit  : synchronous up/down counter (all FFs share clk)
//                      en=1 counts, up=1 counts up, up=0 counts down
// t_ff               : negative-edge triggered T flip-flop (toggle each edge)
// async_counter_4bit : 4-bit ripple (asynchronous) UP counter built from
//                      t_ff cells; clock of stage i+1 is Q of stage i
//==============================================================================
module sync_counter_4bit (
    input            clk,
    input            rst,
    input            en,
    input            up,
    output reg [3:0] q
);
    always @(posedge clk or posedge rst) begin
        if (rst)     q <= 4'd0;
        else if (en) q <= up ? (q + 4'd1) : (q - 4'd1);
    end
endmodule

module t_ff (
    input      clk,
    input      rst,
    output reg q
);
    always @(negedge clk or posedge rst) begin
        if (rst) q <= 1'b0;
        else     q <= ~q;
    end
endmodule

module async_counter_4bit (
    input        clk,
    input        rst,
    output [3:0] q
);
    t_ff ff0 (.clk(clk),  .rst(rst), .q(q[0]));
    t_ff ff1 (.clk(q[0]), .rst(rst), .q(q[1]));
    t_ff ff2 (.clk(q[1]), .rst(rst), .q(q[2]));
    t_ff ff3 (.clk(q[2]), .rst(rst), .q(q[3]));
endmodule
```

Testbench

counters_tb.v

```
`timescale 1ns/1ps
module counters_tb;
    reg        clk, rst, en, up;
    wire [3:0] sq, aq;


    sync_counter_4bit  dut_sync  (.clk(clk), .rst(rst), .en(en), .up(up), .q(sq));
    async_counter_4bit dut_async (.clk(clk), .rst(rst), .q(aq));
```

```
    initial clk = 0;
    always #5 clk = ~clk;


    initial begin
        $dumpfile("counters_tb.vcd"); $dumpvars(0, counters_tb);
        $monitor("t=%0t en=%b up=%b | sync=%d async=%d", $time, en, up, sq, aq);


        rst = 1; en = 0; up = 1;
        #12 rst = 0;


        en = 1; up = 1;         #200;    // count up
        en = 0;                 #30;     // hold (sync only; async always counts)
        en = 1; up = 0;         #100;    // sync counts down
        $finish;
    end
endmodule
```

Run and Observe

```
iverilog -o sim counters_tb.v counters.v && vvp sim
```

Both counters count 0 to 15 and wrap around. While en = 0 the synchronous counter holds its value, but the asynchronous counter has no enable and keeps counting. With up = 0 the synchronous counter counts down.

Result Waveforms

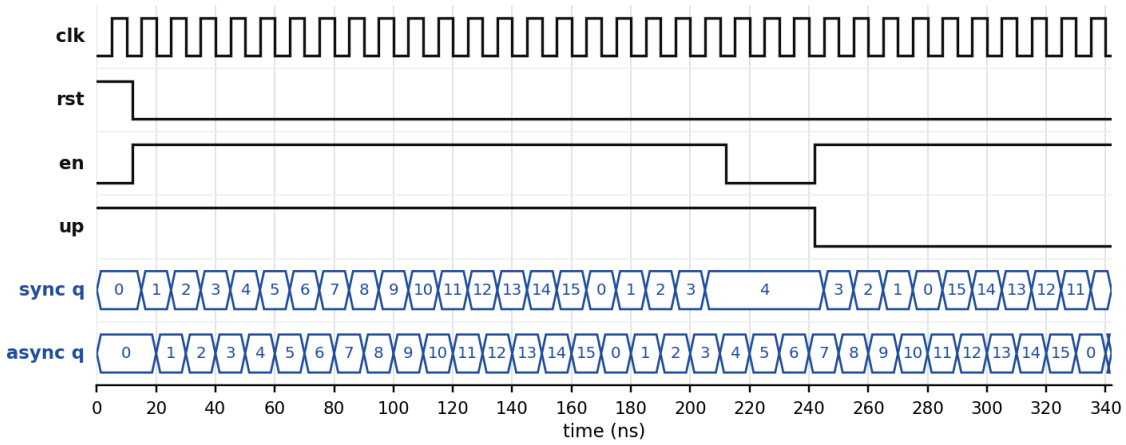


Figure 24: Result waveform of the counters testbench (plotted from a behavioural model of the design)

The synchronous counter updates on rising clock edges: it counts up while en = 1 and up = 1, holds while en = 0, and counts down when up = 0. The asynchronous counter updates on falling clock edges and always counts up.

Question 9: Shift Register

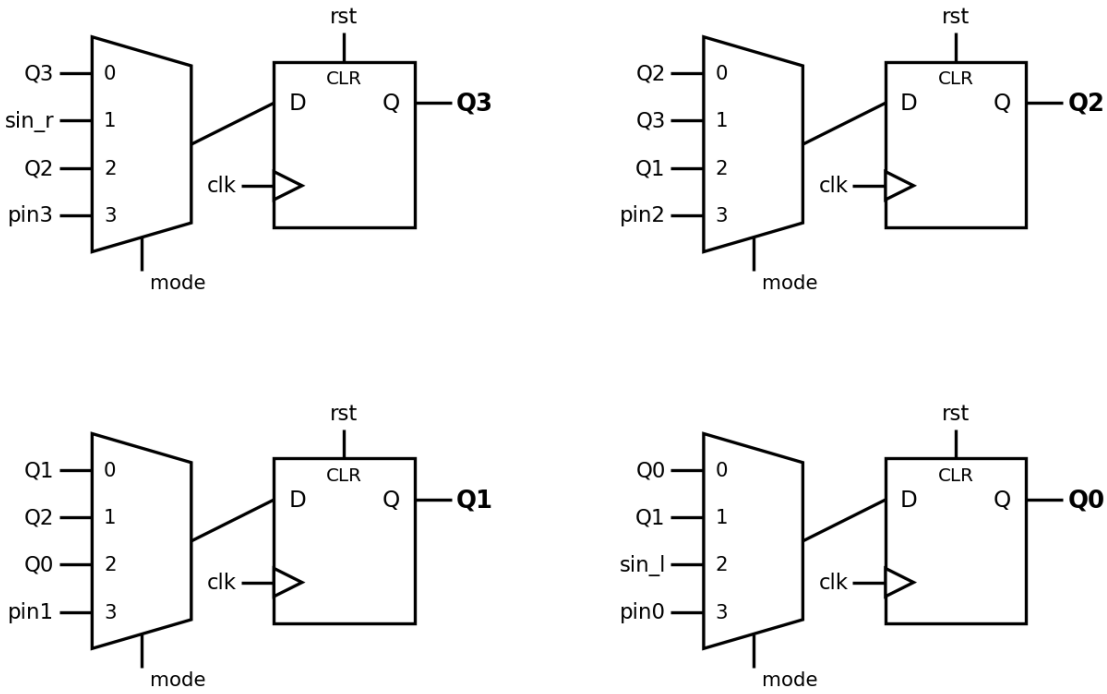
Problem Statement

Design a 4-bit universal shift register that supports hold, shift right, shift left and parallel load, so that it can work as SISO, SIPO, PISO and PIPO. Write a testbench.

Requirements

- Mode 00: hold
- Mode 01: shift right (serial input enters the MSB)
- Mode 10: shift left (serial input enters the LSB)
- Mode 11: parallel load

Design Circuit Diagram



mode: 00 hold 01 shift right 10 shift left 11 parallel load (sout_r = Q0, sout_l = Q3)

Figure 11: 4-bit universal shift register

Function table

mode[1:0]	Operation	Next state Q3 Q2 Q1 Q0
00	Hold	Q3 Q2 Q1 Q0 (no change)
01	Shift right	sin_r Q3 Q2 Q1
10	Shift left	Q2 Q1 Q0 sin_l
11	Parallel load	pin3 pin2 pin1 pin0

Design (Verilog HDL)

shift_register.v

```
`timescale 1ns/1ps
//==============================================================================
```

```
// shift register : N-bit universal shift register (default 4)
//   mode 00 : hold
//   mode 01 : shift right  (sin_r -> MSB, LSB leaves via sout_r)
//   mode 10 : shift left   (sin_l -> LSB, MSB leaves via sout_l)
//   mode 11 : parallel load from pin
// Covers SISO (shift modes), SIPO (read q), PISO (load, then shift), PIPO.
//================================================================================
module shift_register #(parameter N = 4) (
    input              clk,
    input              rst,
    input  [1:0]       mode,
    input              sin_r,
    input              sin_l,
    input  [N-1:0]     pin,
    output reg [N-1:0] q,
    output             sout_r,
    output             sout_l
);
    assign sout_r = q[0];
    assign sout_l = q[N-1];

    always @(posedge clk or posedge rst) begin
        if (rst) q <= {N{1'b0}};
        else begin
            case (mode)
                2'b00:   q <= q;
                2'b01:   q <= {sin_r, q[N-1:1]};
                2'b10:   q <= {q[N-2:0], sin_l};
                2'b11:   q <= pin;
            endcase
        end
    end
endmodule
```

Testbench

shift_register_tb.v

```
`timescale 1ns/1ps
module shift_register_tb;
    reg        clk, rst, sin_r, sin_l;
    reg  [1:0] mode;
    reg  [3:0] pin;
    wire [3:0] q;
    wire       sout_r, sout_l;

    shift_register #(4) dut (.clk(clk), .rst(rst), .mode(mode), .sin_r(sin_r),
                             .sin_l(sin_l), .pin(pin), .q(q),
                             .sout_r(sout_r), .sout_l(sout_l));

    initial clk = 0;
    always #5 clk = ~clk;

    initial begin
        $dumpfile("shift_register_tb.vcd"); $dumpvars(0, shift_register_tb);
        $monitor("t=%0t mode=%b sin_r=%b sin_l=%b pin=%b | q=%b", $time, mode, sin_r, sin_l, pin, q);

        rst = 1; mode = 2'b00; sin_r = 0; sin_l = 0; pin = 4'b0000;
        #12 rst = 0;

        mode = 2'b11; pin = 4'b1010;  #10;                // parallel load 1010
        mode = 2'b00;                 #20;                // hold
        mode = 2'b01; sin_r = 1;      #40;                // shift right, feeding 1s
        mode = 2'b11; pin = 4'b0110;  #10;                // parallel load 0110
        mode = 2'b10; sin_l = 1;      #40;                // shift left, feeding 1s
        mode = 2'b01; sin_r = 0;      #40;                // shift right, feeding 0s
        $finish;
    end
endmodule
```

Run and Observe

```
iverilog -o sim shift_register_tb.v shift_register.v && vvp sim
```

The register is loaded with 1010 and then held. It shifts right while feeding in 1s, is loaded with 0110, shifts left feeding in 1s, and finally shifts right feeding in 0s. Watch q change after each clock.

Result Waveforms

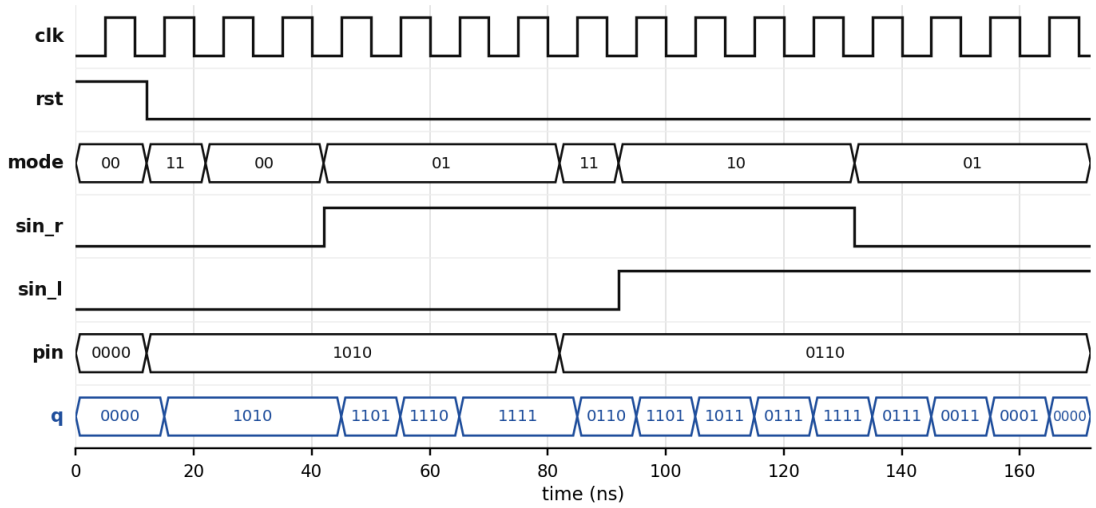


Figure 25: Result waveform of the shift register testbench (plotted from a behavioural model of the design)

q loads 1010, holds, shifts right filling with 1s (1101, 1110, 1111), loads 0110, shifts left filling with 1s (1101, 1011, 0111, 1111), and finally shifts right filling with 0s (0111, 0011, 0001, 0000).

Question 10: Divide-by-N Counter

Problem Statement

Design a divide-by-N counter that divides the input clock frequency by N and produces an output clock with a 50% duty cycle. Write a testbench using one even value and one odd value of N.

Requirements

- Parameterised N (N >= 2)
- Mod-N counter that counts from 0 to N-1
- 50% duty cycle for both even and odd N

Design Circuit Diagram

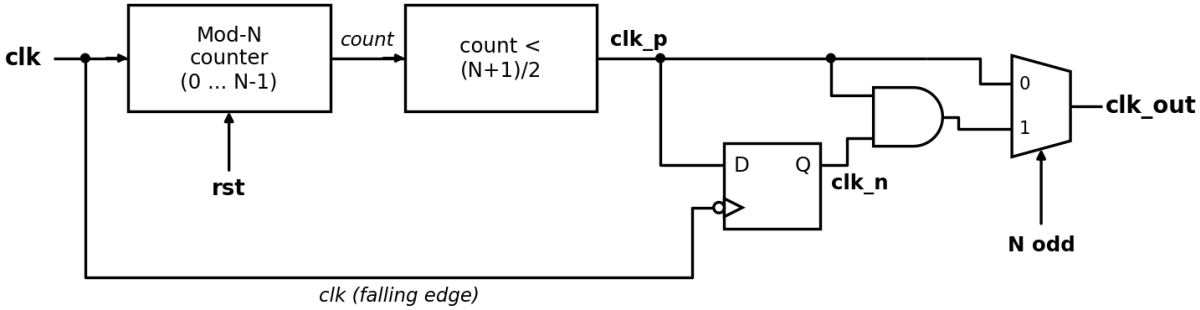
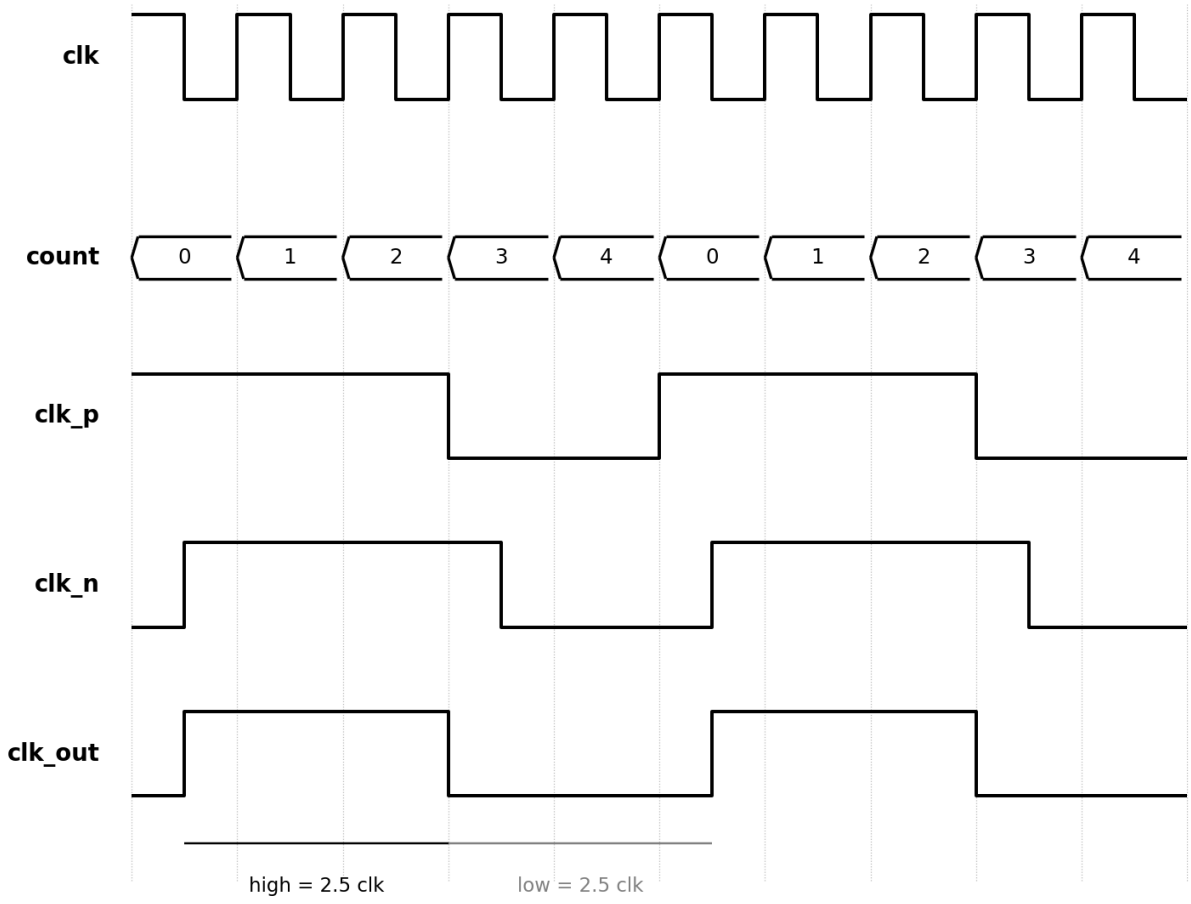


Figure 12: Block diagram of the divide-by-N counter



N = 5: period = 5 clock cycles, duty cycle = 50 %

Figure 13: Timing diagram of the divide-by-5 counter

Output generation

N	clk_out is high when	High time	Period
Even N	count < N/2	N/2 clock cycles	N clock cycles
Odd N	(count < (N+1)/2) AND clk_n	N/2 clock cycles	N clock cycles

Design (Verilog HDL)

div_by_n.v

```
`timescale 1ns/1ps
//==============================================================================
// div_by_n : divide-by-N counter / clock divider, 50 % duty cycle, N >= 2
//   count   : mod-N counter (0 .. N-1) clocked on the rising edge
//   clk_out : clk / N
//   Even N : clk_out = (count < N/2)
//   Odd  N : a rising-edge pulse (count < (N+1)/2) is ANDed with a copy of
//            itself delayed by half a clock (falling-edge register) which
//            trims the high time to exactly N/2 clock periods.
//==============================================================================
module div_by_n #(parameter N = 5, parameter W = 8) (
    input          clk,
    input          rst,
    output         clk_out,
    output reg [W-1:0] count
);
    localparam HIGH = (N + 1) / 2;       // = N/2 for even N


    reg  clk_n;
    wire clk_p = (count < HIGH);

    // mod-N counter on rising edge
    always @(posedge clk or posedge rst) begin
        if (rst)                  count <= {W{1'b0}};
        else if (count == N - 1)  count <= {W{1'b0}};
        else                      count <= count + 1'b1;
    end

    // half-cycle delayed copy (falling edge)
    always @(negedge clk or posedge rst) begin
        if (rst) clk_n <= 1'b0;
        else     clk_n <= clk_p;
    end

    assign clk_out = (N % 2) ? (clk_p & clk_n) : clk_p;
endmodule
```

Testbench

div_by_n_tb.v

```
`timescale 1ns/1ps
module div_by_n_tb;
    reg  clk, rst;
    wire out4, out5;          // divide by 4 (even) and divide by 5 (odd)
    wire [7:0] cnt4, cnt5;

    div_by_n #(4) d4 (.clk(clk), .rst(rst), .clk_out(out4), .count(cnt4));
    div_by_n #(5) d5 (.clk(clk), .rst(rst), .clk_out(out5), .count(cnt5));

    initial clk = 0;
    always #5 clk = ~clk;     // 10 ns clock

    initial begin
        $dumpfile("div_by_n_tb.vcd"); $dumpvars(0, div_by_n_tb);
        $monitor("t=%0t out_div4=%b out_div5=%b", $time, out4, out5);

        rst = 1;
        #12 rst = 0;
        #300;
        $finish;
    end
```

```
endmodule
```

Run and Observe

```
iverilog -o sim div_by_n_tb.v div_by_n.v && vvp sim
```

With a 10 ns clock, out_div4 has a period of 40 ns and out_div5 has a period of 50 ns, both with 50% duty. Verify this in the waveform (GTKWave).

Result Waveforms

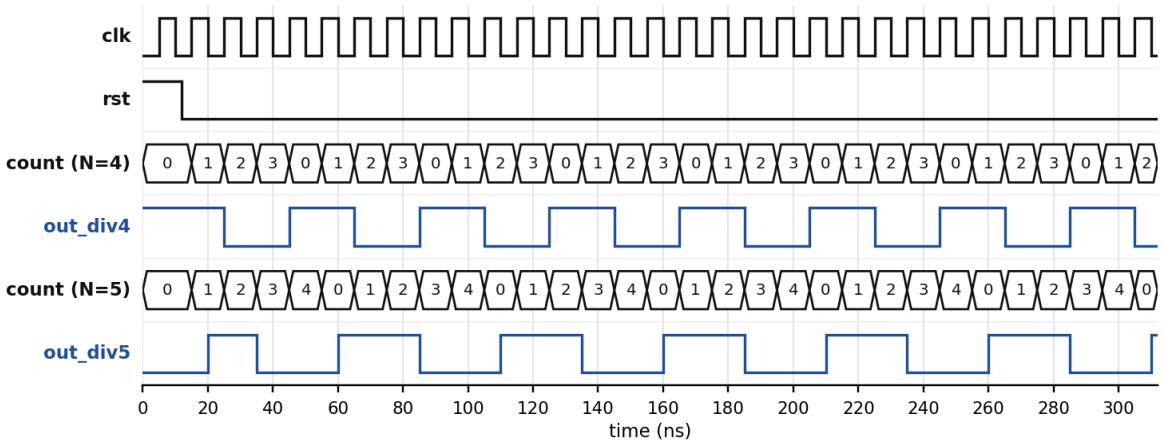


Figure 26: Result waveform of the divide-by-N testbench (plotted from a behavioural model of the design)

out_div4 has a period of 40 ns and out_div5 has a period of 50 ns, both with a 50% duty cycle.

Question 11: Mealy and Moore FSMs

Problem Statement

Design one Mealy FSM and one Moore FSM that detect the overlapping serial bit sequence 1011. The output must go high whenever the pattern 1011 has been received. Write one testbench that drives both machines with the same input stream.

Requirements

- Mealy FSM: 4 states, output depends on state and input
- Moore FSM: 5 states, output depends on state only
- Overlapping detection (for example 1011011 contains two matches)

Design Circuit Diagram

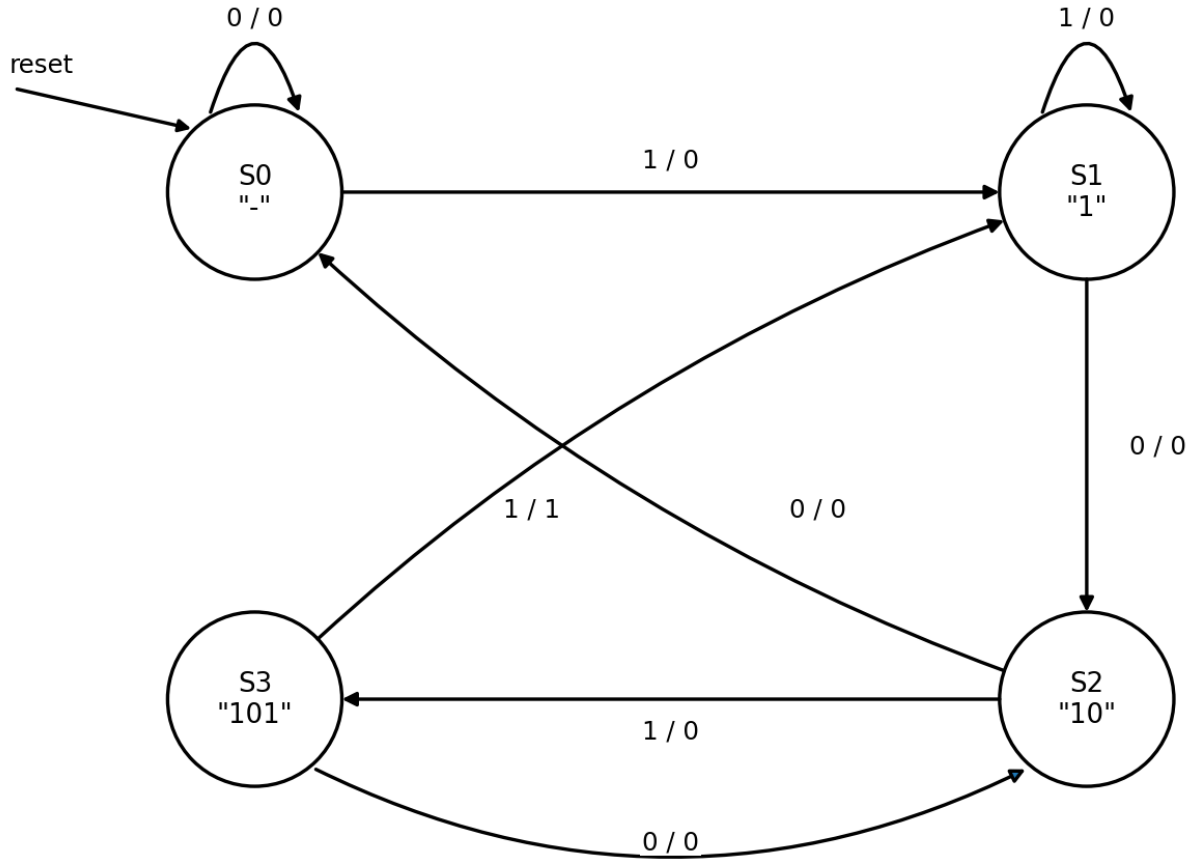


Figure 14: State diagram of the Mealy FSM (1011 detector)

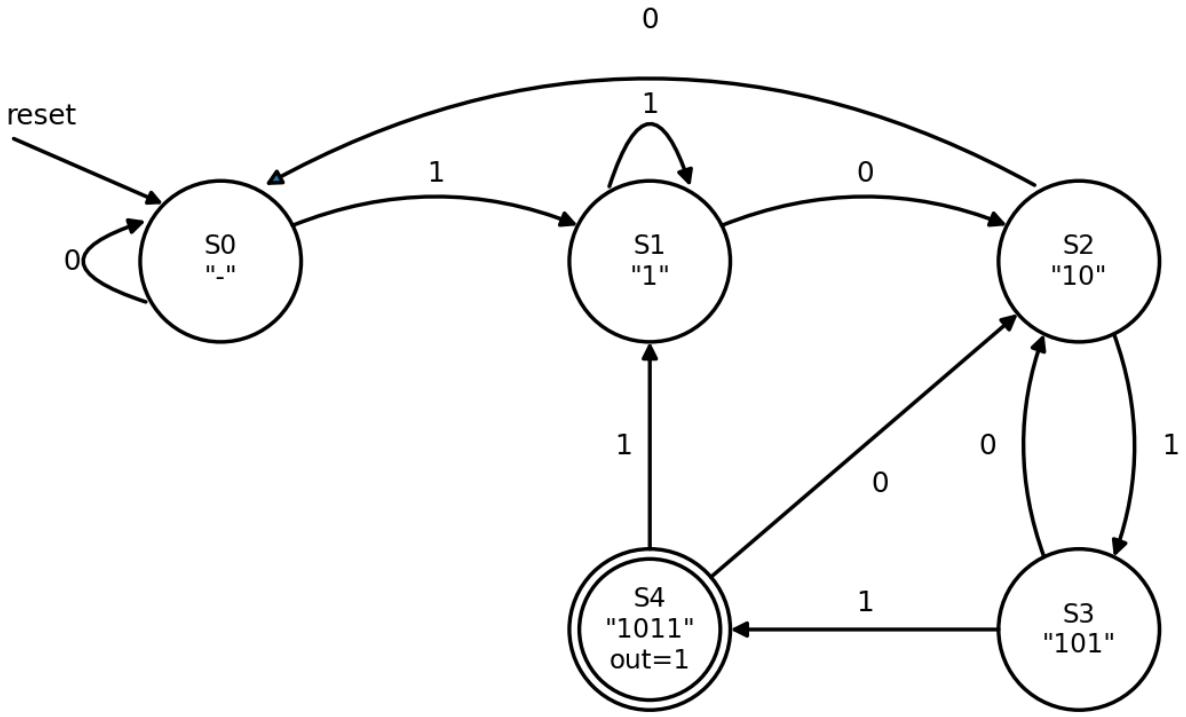


Figure 15: State diagram of the Moore FSM (1011 detector)

State table of the Mealy FSM

Present state	Next state (din = 0)	Next state (din = 1)	Output (din = 0)	Output (din = 1)
S0	S0	S1	0	0
S1	S2	S1	0	0
S2	S0	S3	0	0
S3	S2	S1	0	1

State table of the Moore FSM

Present state	Next state (din = 0)	Next state (din = 1)	Output
S0	S0	S1	0
S1	S2	S1	0
S2	S0	S3	0
S3	S2	S4	0
S4	S2	S1	1

Design (Verilog HDL)

fsm_mealy.v

```
`timescale 1ns/1ps
//================================================================================
// mealy_1011 : Mealy FSM, overlapping "1011" sequence detector
//   Output depends on state AND input -> asserted in the same cycle as the
//   last '1' of the pattern.
//
//   State  Meaning        din=0 / out   din=1 / out
//   S0     no progress    S0 / 0        S1 / 0
//   S1     seen "1"       S2 / 0        S1 / 0
//   S2     seen "10"      S0 / 0        S3 / 0
```

```
//   S3     seen "101"     S2 / 0        S1 / 1
//================================================================================
module mealy_1011 (
    input      clk,
    input      rst,
    input      din,
    output reg dout
);
    localparam S0 = 2'd0, S1 = 2'd1, S2 = 2'd2, S3 = 2'd3;
    reg [1:0] state, next;

    always @(posedge clk or posedge rst) begin      // state register
        if (rst) state <= S0;
        else     state <= next;
    end

    always @(*) begin                               // next-state + output
        next = S0;
        dout = 1'b0;
        case (state)
            S0: next = din ? S1 : S0;
            S1: next = din ? S1 : S2;
            S2: next = din ? S3 : S0;
            S3: begin
                if (din) begin next = S1; dout = 1'b1; end
                else           next = S2;
            end
            default: next = S0;
        endcase
    end
endmodule
```

fsm_moore.v

```
`timescale 1ns/1ps
//================================================================================
// moore_1011 : Moore FSM, overlapping "1011" sequence detector
//   Output depends on state ONLY -> asserted one cycle after the last '1'.
//
//   State  Meaning         din=0   din=1   Output
//   S0     no progress     S0      S1      0
//   S1     seen "1"        S2      S1      0
//   S2     seen "10"       S0      S3      0
//   S3     seen "101"      S2      S4      0
//   S4     seen "1011"     S2      S1      1
//================================================================================
module moore_1011 (
    input  clk,
    input  rst,
    input  din,
    output dout
);
    localparam S0 = 3'd0, S1 = 3'd1, S2 = 3'd2, S3 = 3'd3, S4 = 3'd4;
    reg [2:0] state, next;

    always @(posedge clk or posedge rst) begin      // state register
        if (rst) state <= S0;
        else     state <= next;
    end

    always @(*) begin                               // next-state logic
        case (state)
            S0: next = din ? S1 : S0;
            S1: next = din ? S1 : S2;
            S2: next = din ? S3 : S0;
            S3: next = din ? S4 : S2;
            S4: next = din ? S1 : S2;
            default: next = S0;
        endcase
    end

    assign dout = (state == S4);                    // output logic (Moore)
endmodule
```

Testbench

fsm_tb.v

```
`timescale 1ns/1ps
module fsm_tb;
    reg  clk, rst, din;
    wire mealy_out, moore_out;
    reg  [15:0] stream;
    integer i;

    mealy_1011 u_mealy (.clk(clk), .rst(rst), .din(din), .dout(mealy_out));
    moore_1011 u_moore (.clk(clk), .rst(rst), .din(din), .dout(moore_out));

    initial clk = 0;
    always #5 clk = ~clk;

    initial begin
        $dumpfile("fsm_tb.vcd"); $dumpvars(0, fsm_tb);
        $monitor("t=%0t din=%b | mealy=%b moore=%b", $time, din, mealy_out, moore_out);

        stream = 16'b1011 0110 1011 1011;     // contains several 1011 patterns
        rst = 1; din = 0;
        #12 rst = 0;

        for (i = 15; i >= 0; i = i - 1) begin
            @(negedge clk);
            din = stream[i];
        end
        @(negedge clk); din = 0;
        #20;
        $finish;
    end
endmodule
```

Run and Observe

```
iverilog -o sim fsm_tb.v fsm_mealy.v fsm_moore.v && vvp sim
```

The input stream 1011 0110 1011 1011 is applied. The Mealy output goes high in the same clock as the last 1 of each pattern; the Moore output goes high one clock later.

Result Waveforms

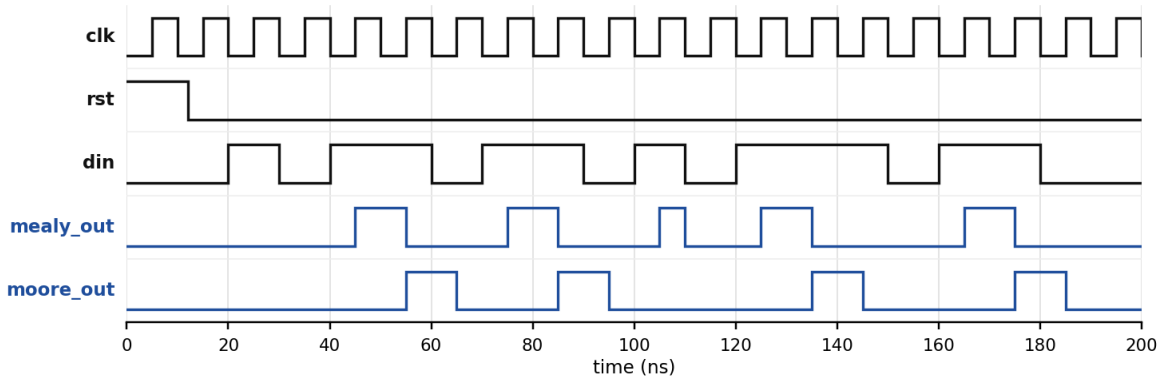


Figure 27: Result waveform of the Mealy and Moore FSM testbench (plotted from a behavioural model of the design)

Both machines detect the four 1011 patterns in the input stream (the Moore output goes high after the clock edges at 55, 85, 135 and 175 ns). The Mealy output depends on the current input, so it can also show a short pulse between a rising clock edge and the next input change (for example near 105 ns); only its value at the rising clock edge is the detection result.

Question 12: Random Sequence Generator (LFSR)

Problem Statement

Design a 4-bit pseudo-random sequence generator using a Linear Feedback Shift Register (LFSR) with the polynomial x^4 + x^3 + 1. Write a testbench to show the generated sequence.

Requirements

- Seed (reset) value 0001
- Feedback = Q3 XOR Q2
- Maximal length: 15 distinct non-zero states

Design Circuit Diagram

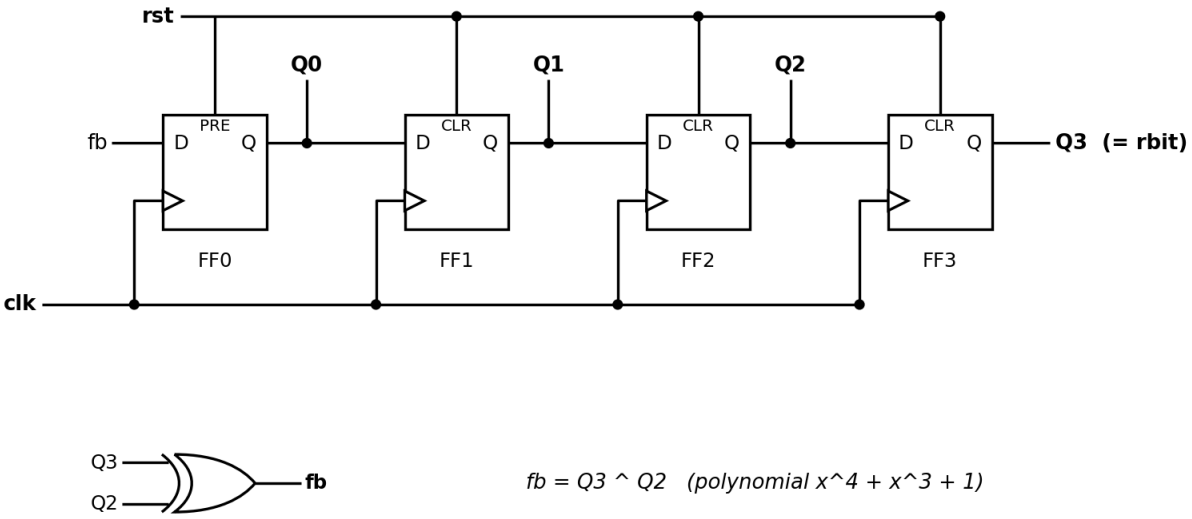


Figure 16: 4-bit LFSR random sequence generator (x^4 + x^3 + 1)

State sequence (seed 0001, feedback = Q3 XOR Q2; clock pulse 15 returns to the seed)

Clock pulse	Q3 Q2 Q1 Q0	Decimal	Clock pulse	Q3 Q2 Q1 Q0	Decimal
0	0001	1	8	0101	5
1	0010	2	9	1011	11
2	0100	4	10	0111	7
3	1001	9	11	1111	15
4	0011	3	12	1110	14
5	0110	6	13	1100	12
6	1101	13	14	1000	8
7	1010	10	15	0001	1

Design (Verilog HDL)

lfsr.v

```
`timescale 1ns/1ps
//==============================================================================
// lfsr_4bit : 4-bit Fibonacci LFSR, polynomial x^4 + x^3 + 1
//   feedback = q[3] ^ q[2]; shifts left each clock; seed (reset) = 0001
//   Maximal length: cycles through all 15 non-zero states.
//   'rbit' is the serial pseudo-random bit stream (q[3]).
//==============================================================================
module lfsr_4bit (
    input            clk,
    input            rst,
    output reg [3:0] q,
```

```
    output           rbit
);
    wire fb = q[3] ^ q[2];
    assign rbit = q[3];

    always @(posedge clk or posedge rst) begin
        if (rst) q <= 4'b0001;
        else     q <= {q[2:0], fb};
    end
endmodule
```

Testbench

lfsr_tb.v

```
`timescale 1ns/1ps
module lfsr_tb;
    reg        clk, rst;
    wire [3:0] q;
    wire       rbit;

    lfsr_4bit dut (.clk(clk), .rst(rst), .q(q), .rbit(rbit));

    initial clk = 0;
    always #5 clk = ~clk;

    initial begin
        $dumpfile("lfsr_tb.vcd"); $dumpvars(0, lfsr_tb);
        $monitor("t=%0t q=%b (%d) rbit=%b", $time, q, q, rbit);

        rst = 1;
        #12 rst = 0;
        #180;                  // a bit more than 15 clocks, so you can see it repeat
        $finish;
    end
endmodule
```

Run and Observe

`iverilog -o sim lfsr_tb.v lfsr.v && vvp sim`The sequence in decimal is 1, 2, 4, 9, 3, 6, 13, 10, 5, 11, 7, 15, 14, 12, 8 and then it repeats.

Result Waveforms

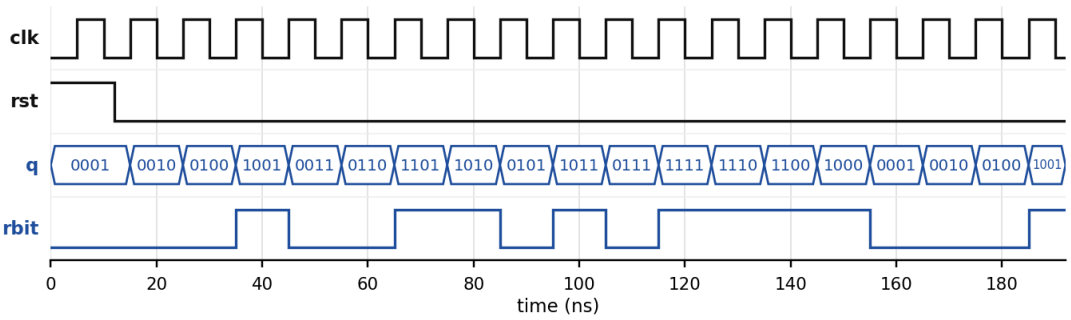


Figure 28: Result waveform of the LFSR testbench (plotted from a behavioural model of the design)

q follows 1, 2, 4, 9, 3, 6, 13, 10, 5, 11, 7, 15, 14, 12, 8 and then returns to 1 after 15 clock cycles. rbit is the MSB of q.